# Computer System Architecture

Lecture 4 - Arithmetic and Logic Unit

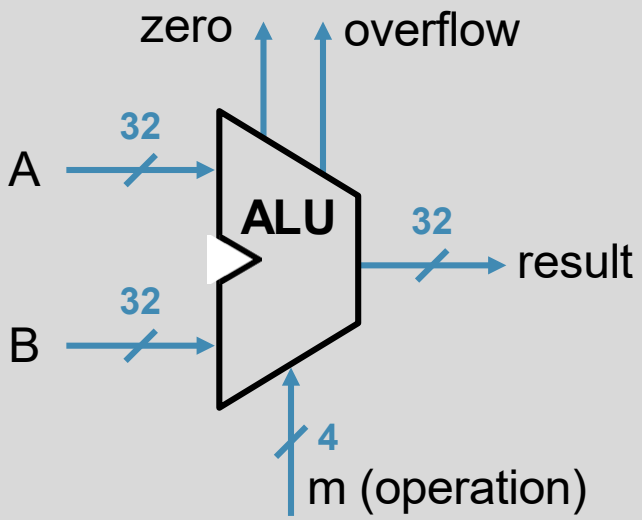
October 2026

# Arithmetic for Computers

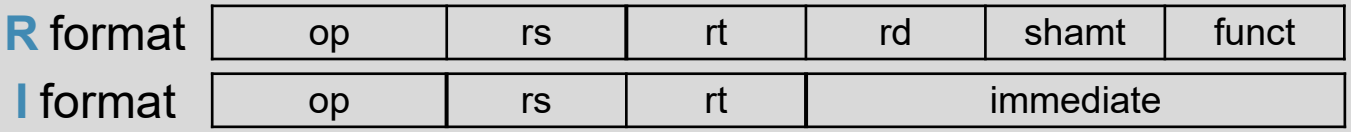
- Operations on integers
  - Addition and subtraction
  - Multiplication and division
  - Dealing with overflow
- Floating-point real numbers
  - Representation and operations

# MIPS Arithmetic and Logic Unit (ALU)

- Must support the Arithmetic/Logic operations of the ISA
  - `add`, `addi`, `addiu`, `addu`
  - `sub`, `subu`
  - `mult`, `multu`, `div`, `divu`
  - `sqrt`
  - `and`, `andi`, `nor`, `or`, `ori`, `xor`, `xori`
  - `beq`, `bne`, `slt`, `slti`, `sltiu`, `sltu`
- With special handling for
  - sign extend – `addi`, `addiu`, `slti`, `sltiu`
  - zero extend – `andi`, `ori`, `xori`
  - overflow detection – `add`, `addi`, `sub`



# MIPS Arithmetic and Logic Instructions




## R-format

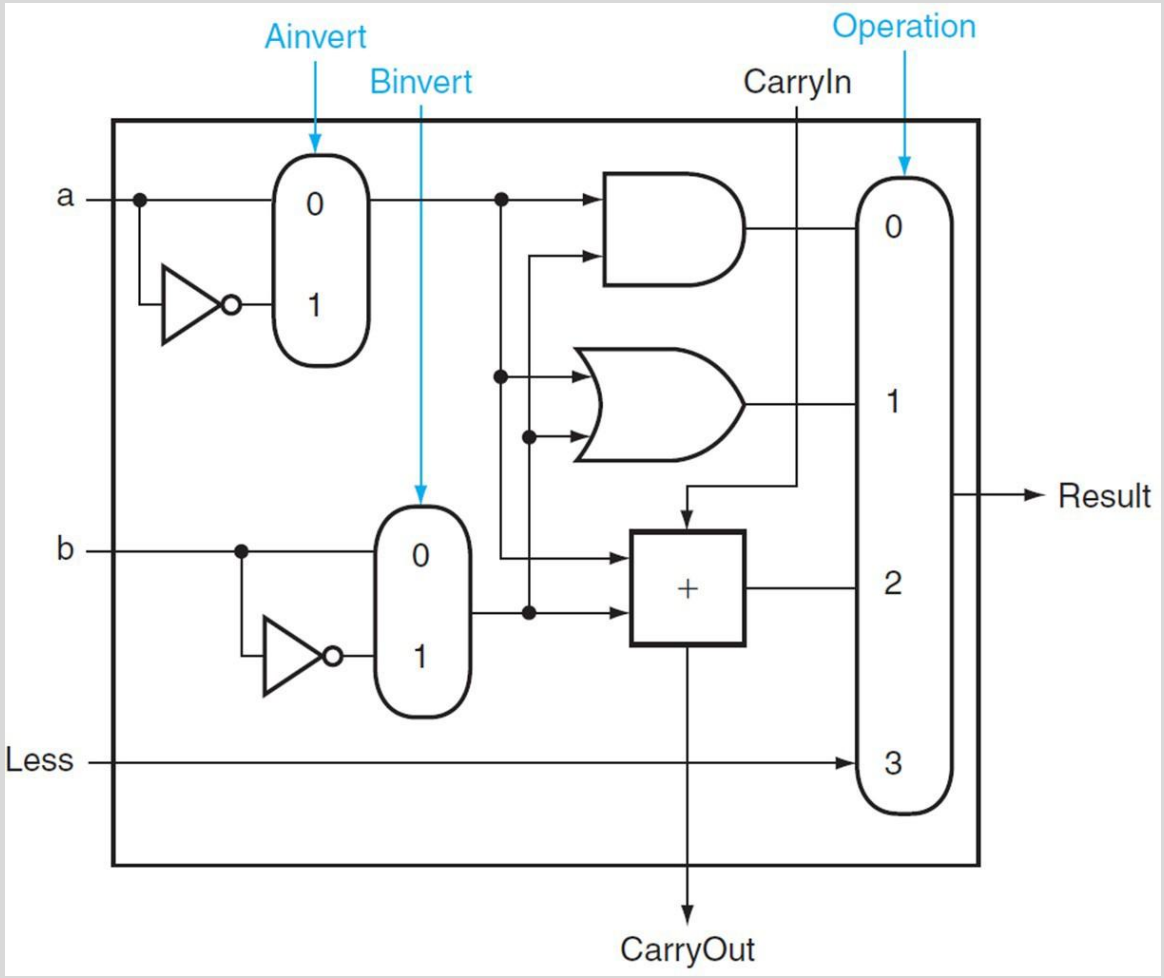
| | op | funct |
|---|---|---|
| add | 000000 | 100000 |
| addu | 000000 | 100001 |
| sub | 000000 | 100010 |
| subu | 000000 | 100011 |
| and | 000000 | 100100 |
| or | 000000 | 100101 |
| xor | 000000 | 100110 |
| nor | 000000 | 100111 |

| | op | funct |
|---|---|---|
| slt | 000000 | 101010 |
| sltu | 000000 | 101011 |

## I-format

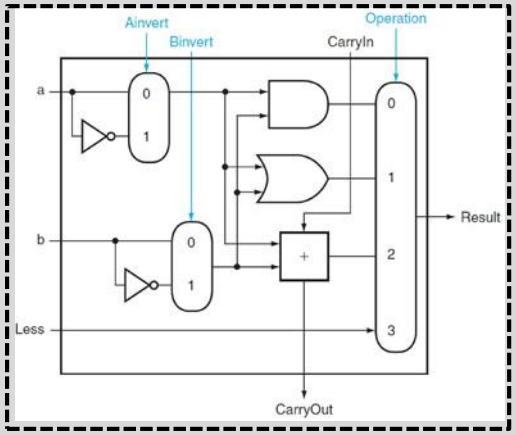
| | op |
|---|---|
| addi | 001000 |
| addiu | 001001 |
| slti | 001010 |
| sltiu | 001011 |
| andi | 001100 |
| ori | 001101 |
| xori | 001110 |
| lui | 001111 |

# (Simplyfied) 1-bit MIPS ALU

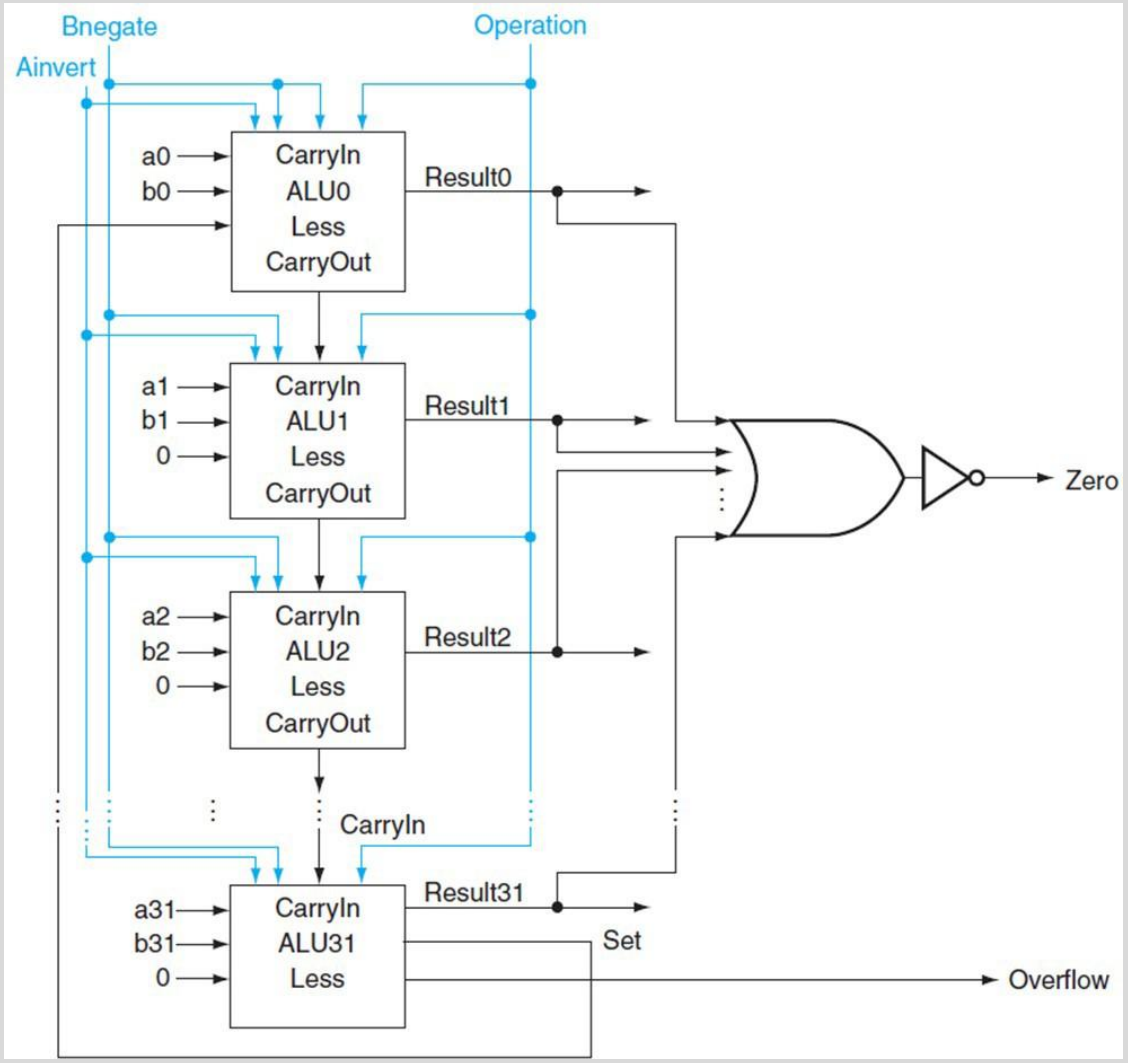



- `and`, `or`, `nor`, `add`, `sub`, `slt`

# Final 32-bit ALU

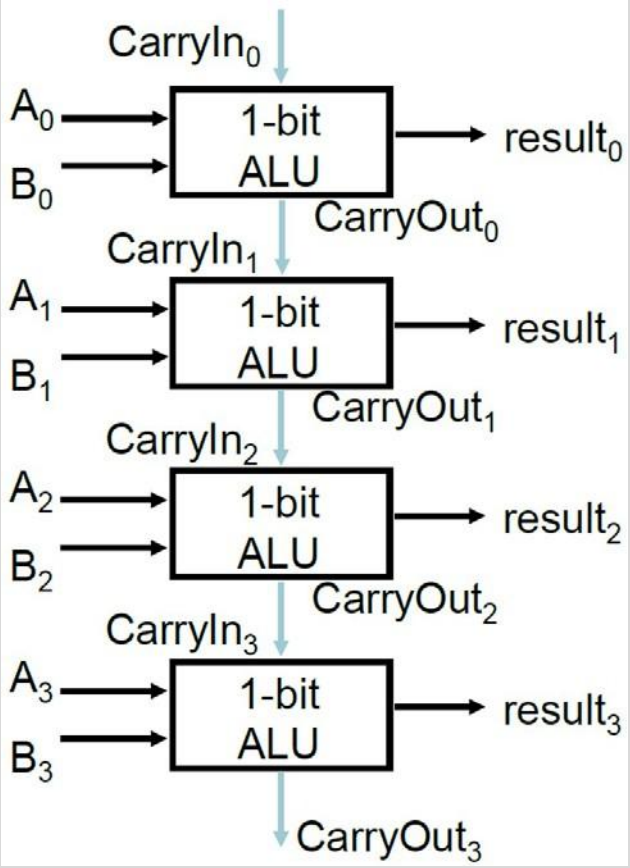



1-bit ALU

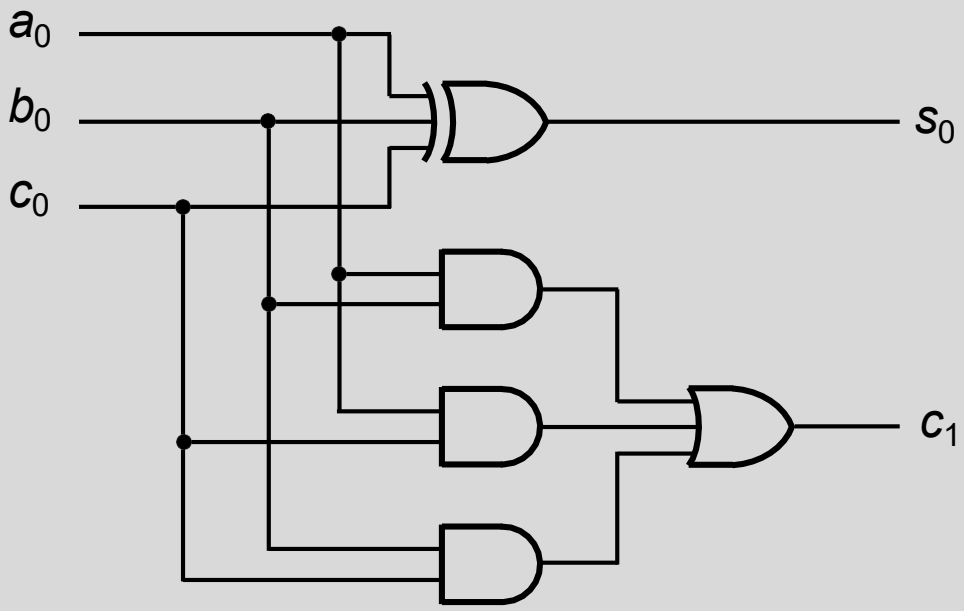


# Performance Issues

- Critical path of *n*-bit ripple-carry adder is *n**CP
  - Wait for carry out
- Design trick (use hardware)
  - Carry-lookahead adder



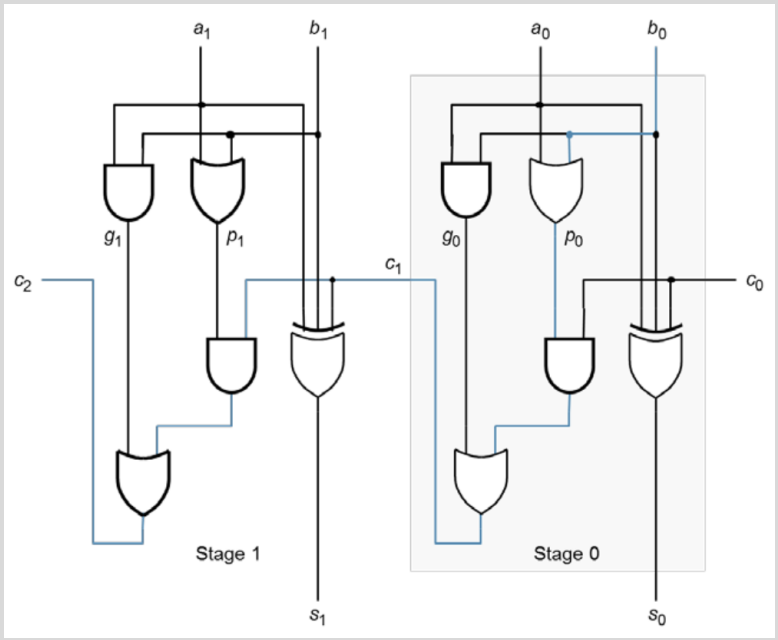
# Ripple-Carry vs Carry-Lookahead Adder



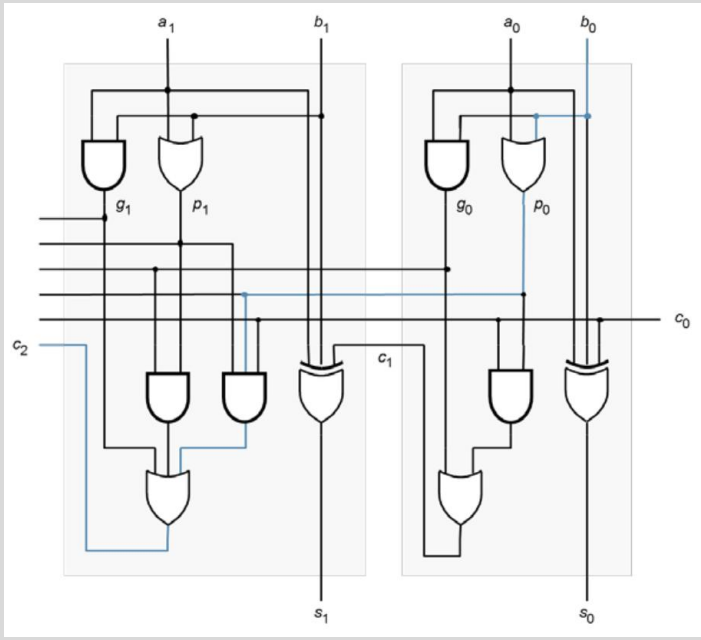

$$s_0 = a_0 \oplus b_0 \oplus c_0$$

$$c_1 = a_0 b_0 + a_0 c_0 + b_0 c_0$$

# Ripple-Carry vs Carry-Lookahead Adder




Ripple-Carry Adder

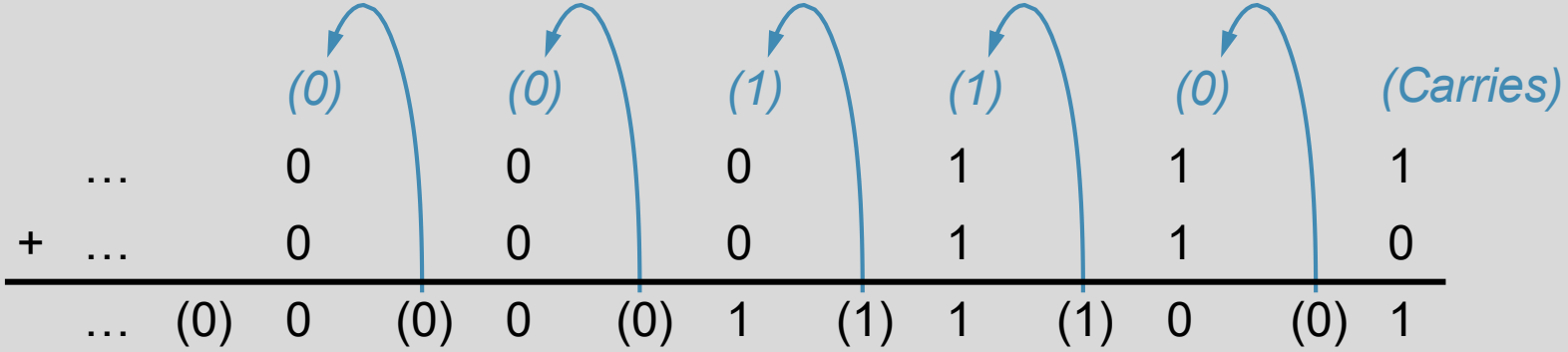



Carry-Lookahead Adder

$$c_1 = a_0 b_0 + (a_0 + b_0) c_0$$
$$c_1 = g_0 + p_0 c_0$$
$$c_2 = g_1 + p_1 g_0 + p_1 p_0 c_0$$

# Integer Addition

- **Example:** 7 + 6




- **Overflow** if result out of range
  - Adding positive and negative operands, no overflow
  - Adding two positive operands
    - Overflow if result sign is 1
  - Adding two negative operands
    - Overflow if result sign is 0

# Integer Subtraction

- Add negation of second operand
- **Example:** 7 – 6 = 7 + (–6)

| | | |
|---|---|---|
| | +7: | 0000 0000 … 0000 0111 |
| + | –6: | 1111 1111 … 1111 1010 |
| | +1: | 0000 0000 … 0000 0001 |

- Overflow if result out of range
  - Subtracting 2 positive or 2 negative operands, no overflow
  - Subtracting positive from negative operand
    - Overflow if result sign is 0
  - Subtracting negative from positive operand
    - Overflow if result sign is 1

# Dealing with Overflow

- Overflow occurs when the result of an operation cannot be represented in 32-bits
  - Sign bit contains a **value** bit of the result

| Operation | Operand A | Operand B | Result indicating overflow |
|---|---|---|---|
| A + B | ≥ 0 | ≥ 0 | < 0 |
| A + B | < 0 | < 0 | ≥ 0 |
| A − B | ≥ 0 | < 0 | < 0 |
| A − B | < 0 | ≥ 0 | ≥ 0 |

- MIPS signifies overflow with an exception (interrupt)
  - An unscheduled procedure call
  - Exception program counter (EPC) saves the address

# Dealing with Overflow

- Some languages (e.g., C) ignore overflow
  - Use MIPS `addu`, `addui`, `subu` instructions
- Other languages (e.g., Ada, Fortran) require raising an exception
  - Use MIPS `add`, `addi`, `sub` instructions
  - On overflow, invoke exception handler
  - Save PC in EPC
  - Jump to predefined handler address
  - Instruction `mfc0` retrieves EPC value to return after corrective action

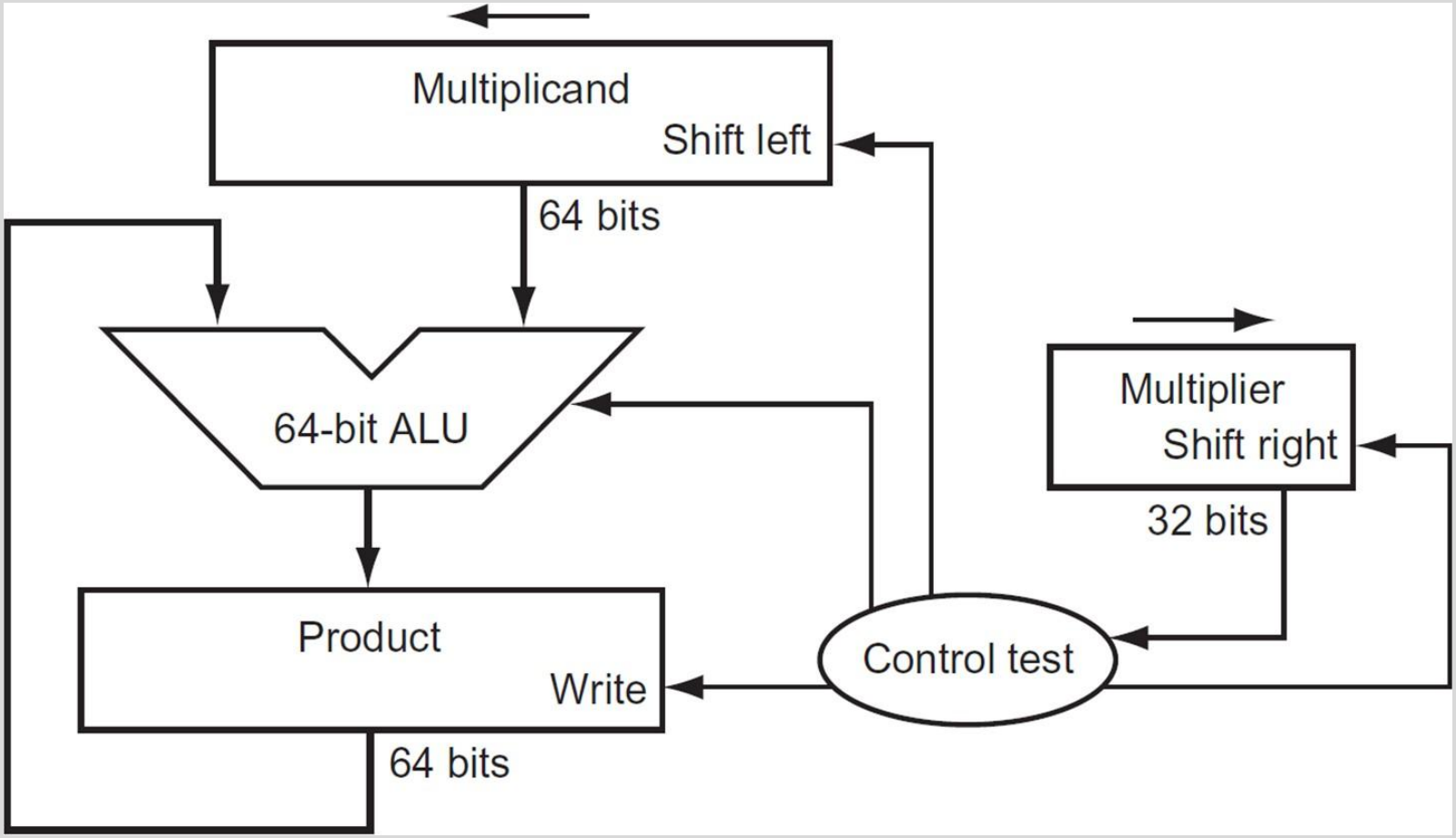
# Multiplication

- More complicated than addition
  - Can be accomplished via **shifting** and **adding**

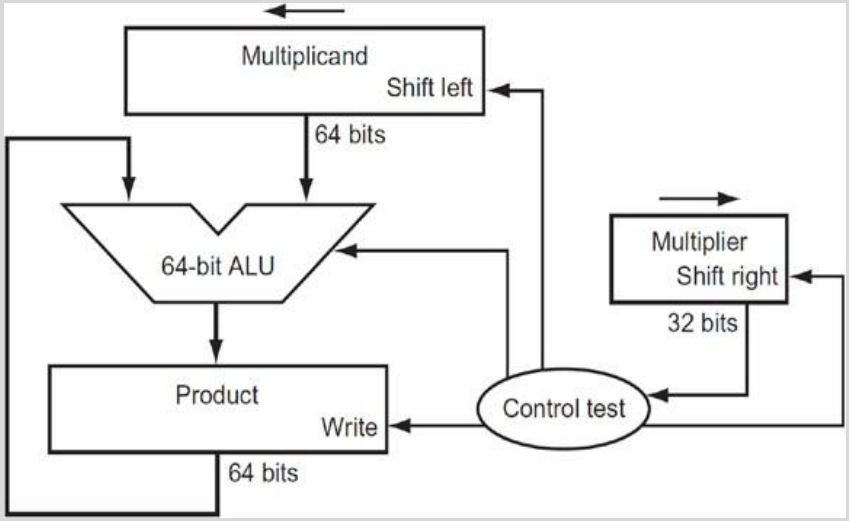
```
      0010        (multiplicand)
×     1011        (multiplier)
-------------
      0010
     0010
    0000          (partial product array)
   0010
-------------
  00010110        (product)
```

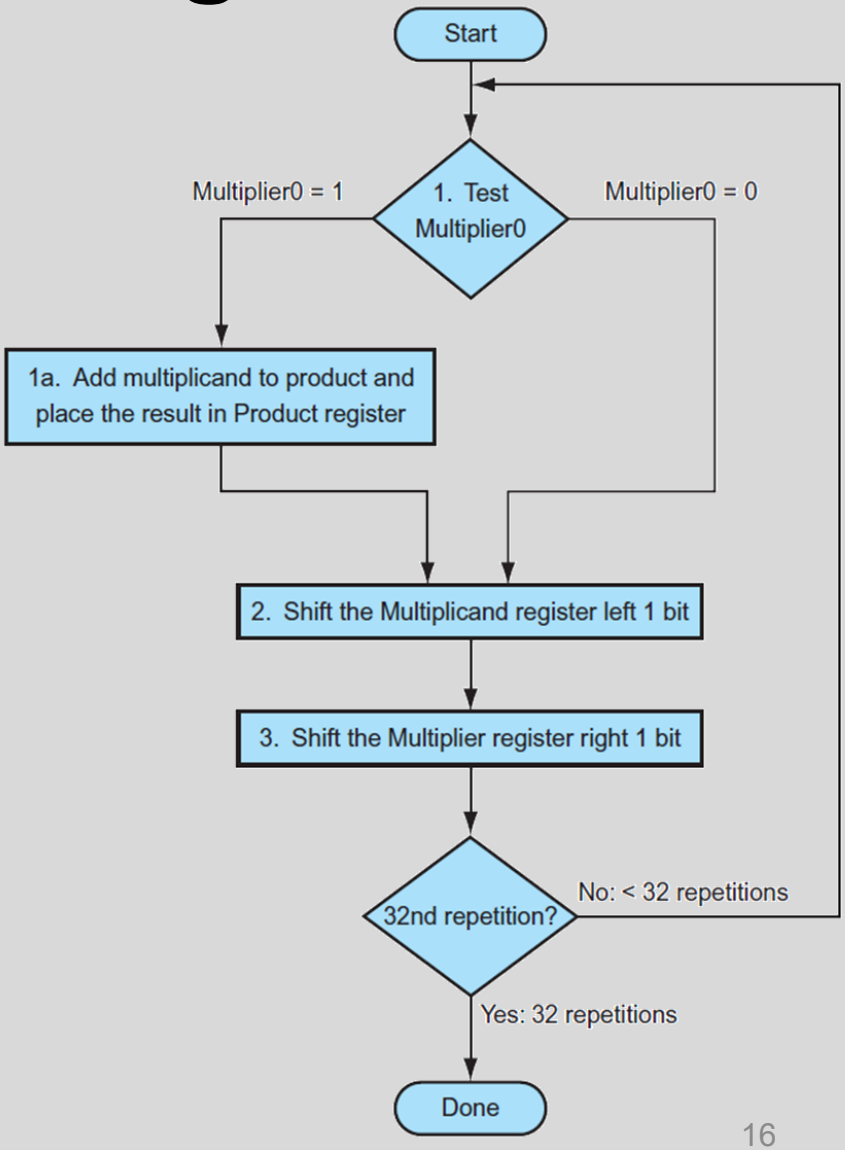
- **Double precision** product is produced
- More time and more area is required

# Multiplication Hardware (1st Version)



# Multiplication Algorithm

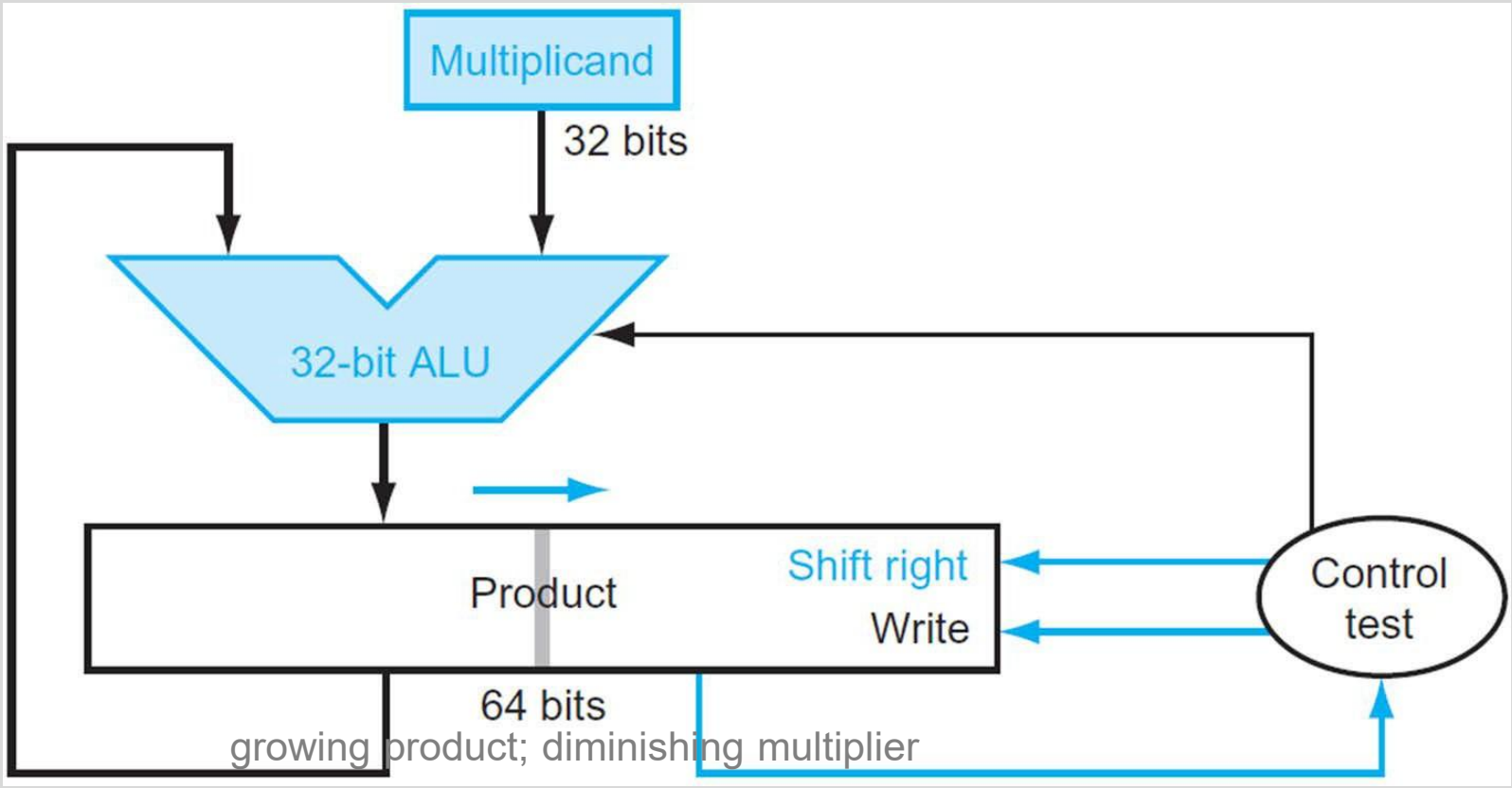


# Exercise

- Calculate 0010×0011

# Refined Multiplication Hardware

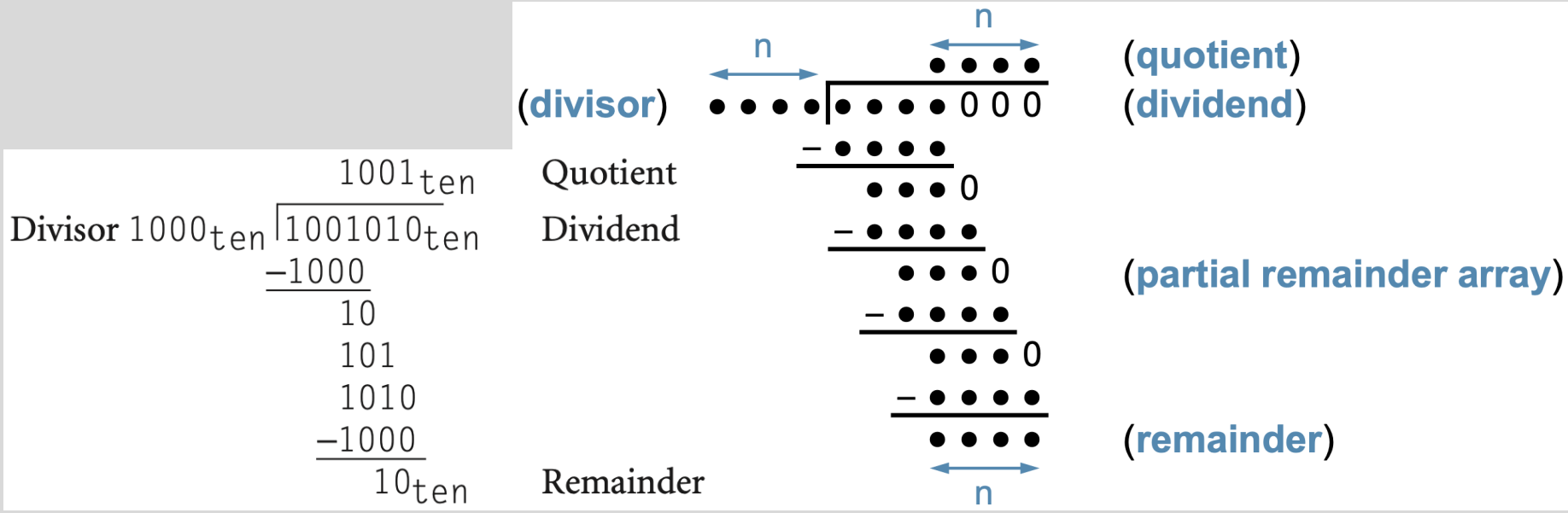


# MIPS Multiplication Instruction

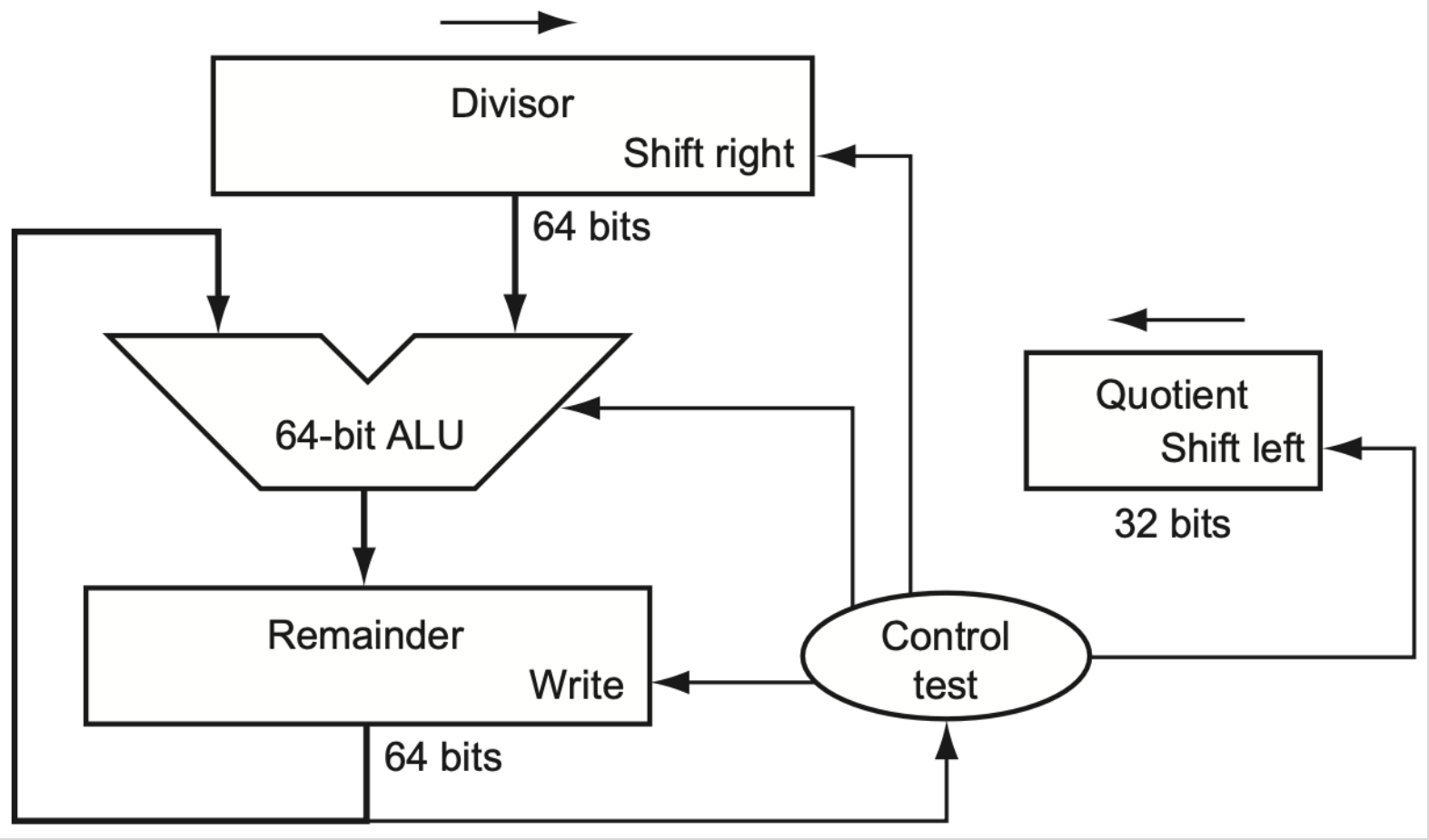
- Two 32-bit registers for product
  - HI: most-significant 32 bits
  - LO: least-significant 32-bits
- Instructions
  - `mult rs, rt` / `multu rs, rt`
    - 64-bit product in HI/LO
  - `mfhi rd` / `mflo rd`
    - Move from HI/LO to rd
    - Can test HI value to see if product overflows 32 bits
  - `mul rd, rs, rt`
    - Least-significant 32 bits of product ➔ rd

# Division

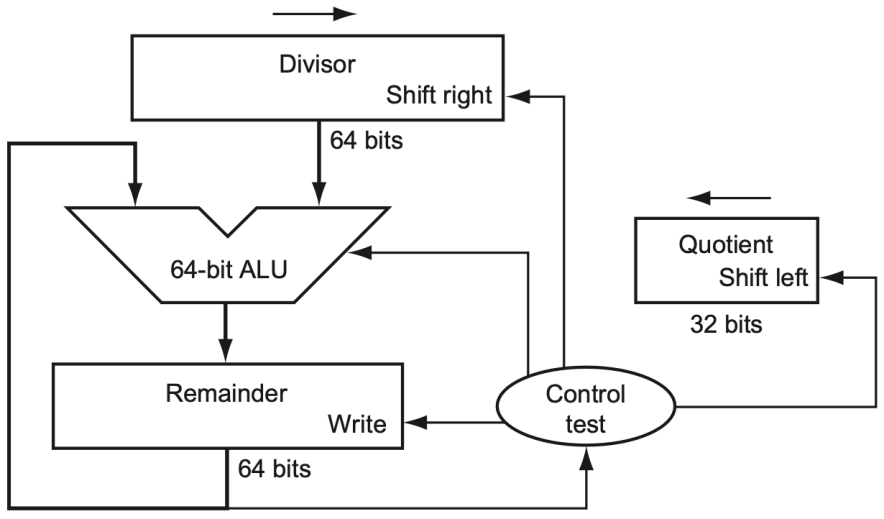
- Dividend = Quotient × Divisor + Remainder

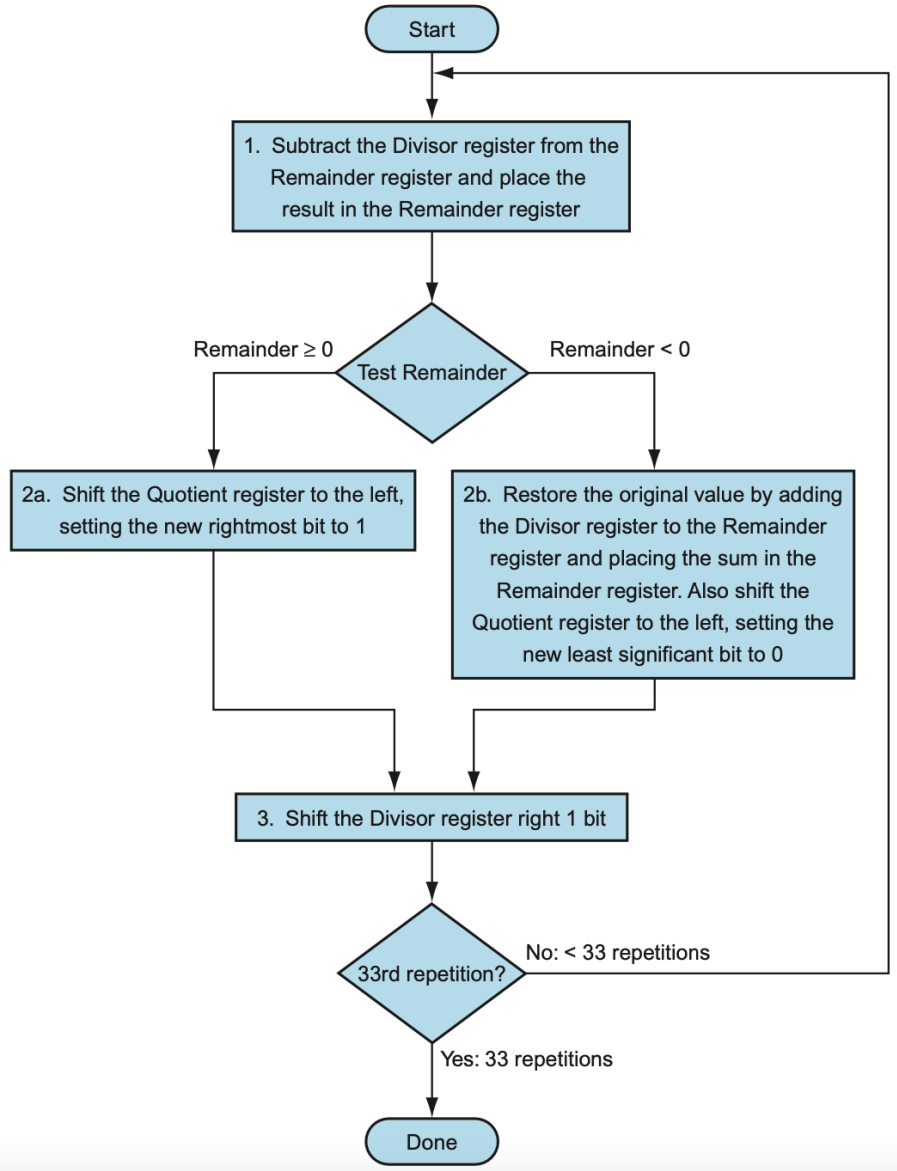


# Division Hardware (1st Version)



# Division Algorithm

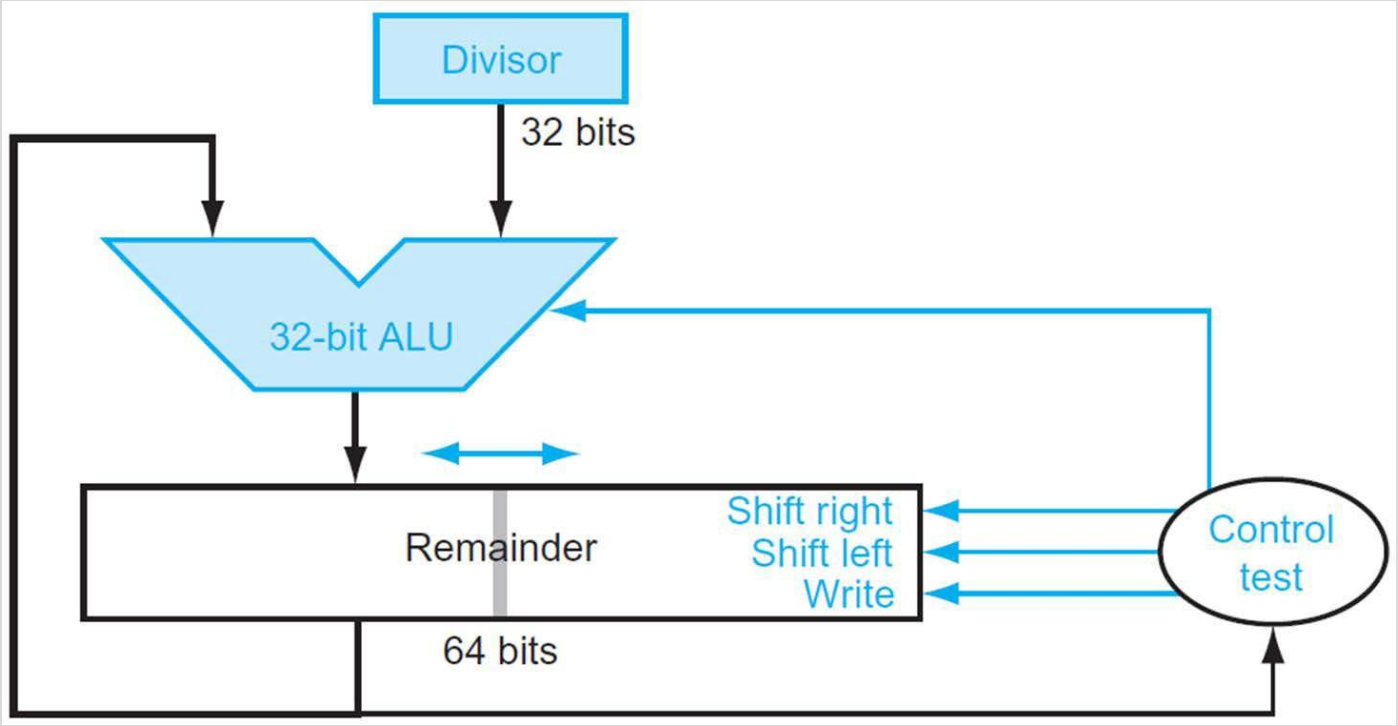


# Exercise

- Divide 00000111 by 0010

# Optimized Divider




- One cycle per partial-remainder subtraction
- Looks a lot like a multiplier!
  - Same hardware can be used for both

# MIPS Division Instruction

- Use HI/LO registers for result
  - HI: 32-bit remainder
  - LO: 32-bit quotient
- Instructions
  - `div rs, rt` / `divu rs, rt`
  - No overflow or divide-by-0 checking
    - Software must perform checks if required
- Use `mfhi`, `mflo` to access result

# Floating-Point Numbers

- Representation for non-integral numbers
  - Including very small and very large numbers
- Like scientific notation
  - $-2.34 \times 10^{56}$ (**normalized**)
  - $+0.002 \times 10^{-4}$ (**not normalized**)
  - $+987.02 \times 10^{9}$ (**not normalized**)
- In binary
  - $\pm 1.xxxxxxx_2 \times 2^{yyyy}$
- Types `float` and `double` in C

# Floating-Point Standard

- Defined by IEEE 754-1985 standard
- Developed in response to divergence of representations
  - Portability issues for scientific code
- Now almost universally adopted
- Two representations
  - Single precision (32-bit)
  - Double precision (64-bit)

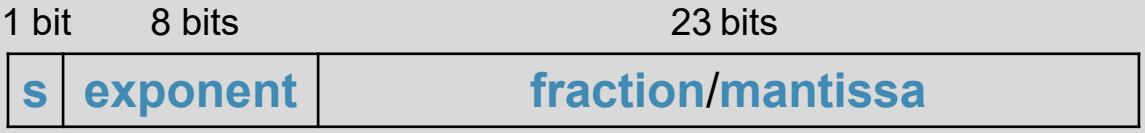
# IEEE Floating-Point Format (Single-Precision)




$$x = (-1)^s \times (1+\text{fraction}) \times 2^{(\text{exponent}-127)}$$

- s: sign bit (0 → non-negative, 1 → negative)
- Normalized significand:
  - 1+fraction
- exponent:
  - All numbers are shifted up by 127 to be stored (become unsigned numbers)
    - True exponent = 0 ➔ exponent = 127
    - True exponent = −127 ➔ exponent = 0 (reserved)
    - True exponent = 127 ➔ exponent = 254

# IEEE Floating-Point Format




**Single-Precision** $x = (-1)^s \times (1+\text{fraction}) \times 2^{(\text{exponent}-127)}$

| exponent | fraction | Value represented |
|---|---|---|
| 0000 0000 | 0 | 0 |
| 1111 1111 | 0 | ±∞ |
| 1111 1111 | non-zero | NaN |

---




**Double-Precision** $x = (-1)^s \times (1+\text{fraction}) \times 2^{(\text{exponent}-1023)}$

# Exercise

- How to represent –0.75 in IEEE single and double precision binary floating-point format?

# Floating-Point Addition

- Consider a 4-digit decimal example
  - $9.999 \times 10^{1} + 1.610 \times 10^{-1}$
- 1. Align decimal points
  - Shift number with smaller exponent
  - $9.999 \times 10^{1} + 0.016 \times 10^{1}$
- 2. Add significands
  - $9.999 \times 10^{1} + 0.016 \times 10^{1} = 10.015 \times 10^{1}$
- 3. Normalize result & check for over/underflow
  - $1.0015 \times 10^{2}$
- 4. Round and renormalize if necessary
  - $1.002 \times 10^{2}$

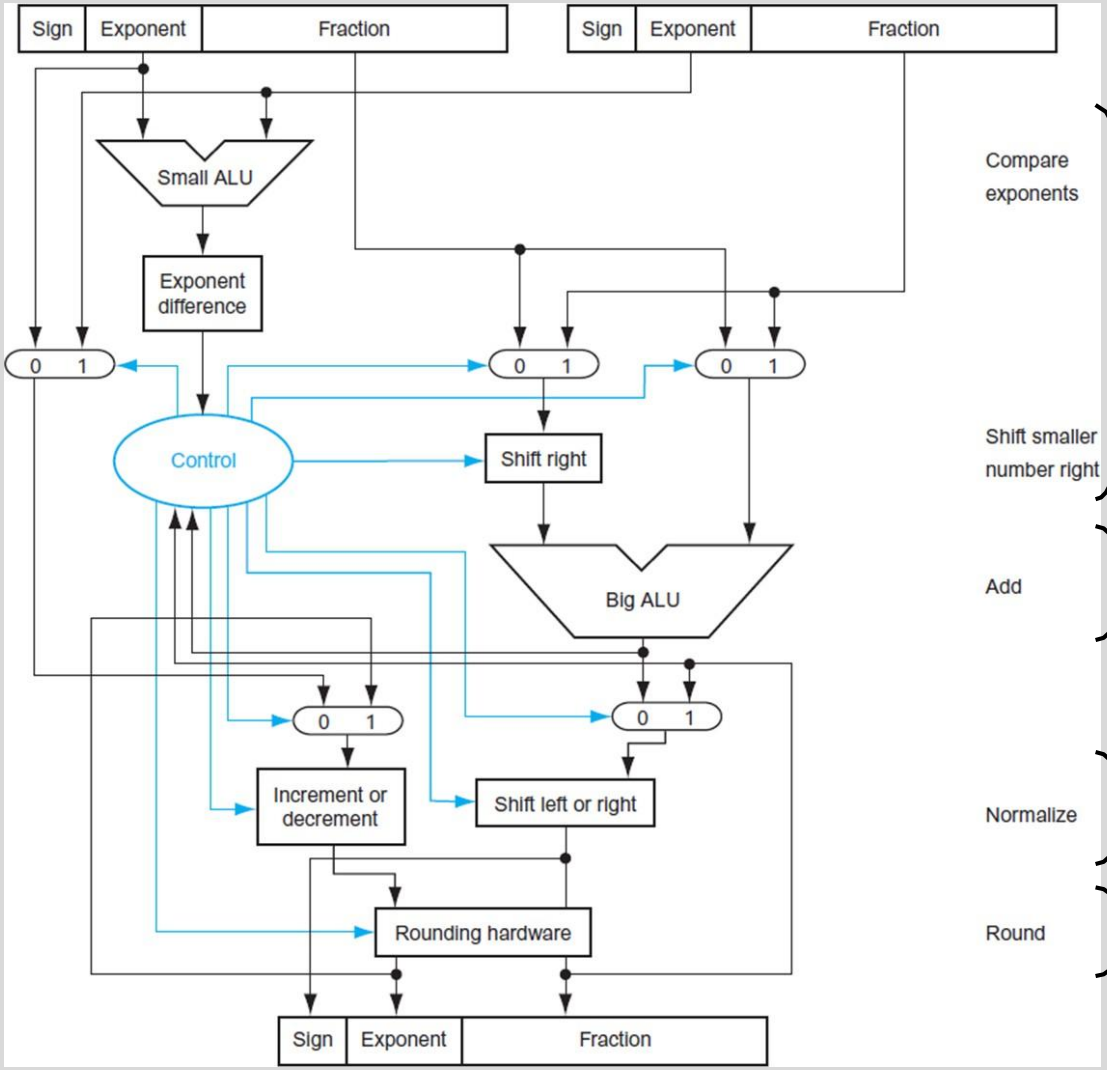
# Floating-Point Addition

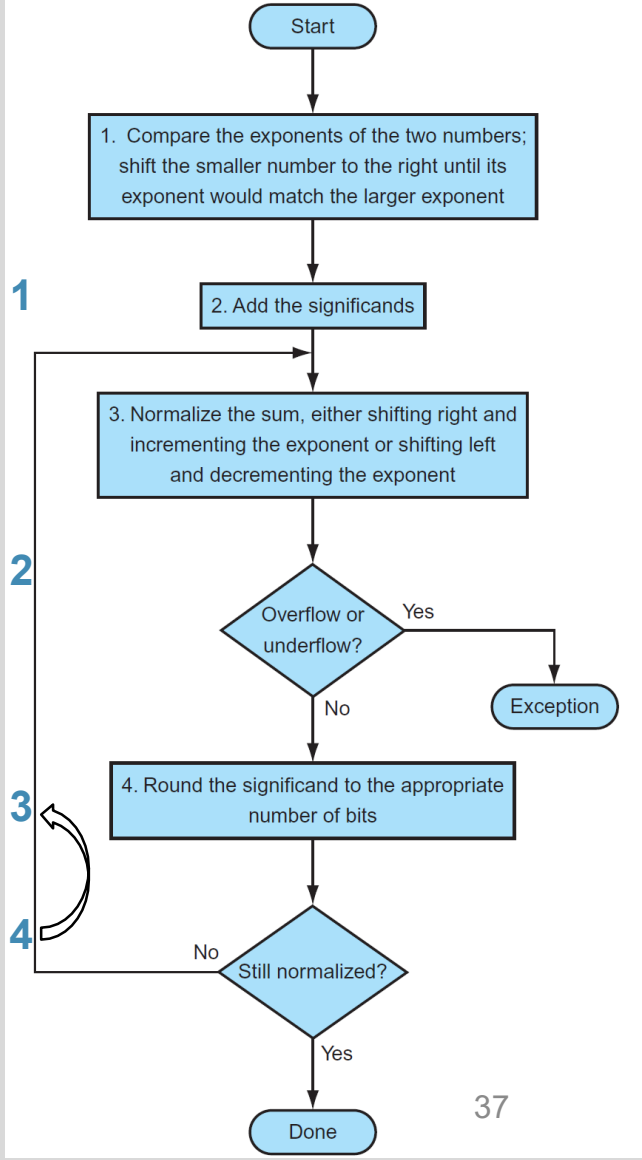
- Now consider a 4-digit binary example
  - $1.000_2 \times 2^{-1} + -1.110_2 \times 2^{-2}$ (0.5 + –0.4375)
- 1. Align binary points
  - Shift number with smaller exponent
  - $1.000_2 \times 2^{-1} + -0.111_2 \times 2^{-1}$
- 2. Add significands
  - $1.000_2 \times 2^{-1} + -0.111_2 \times 2^{-1} = 0.001_2 \times 2^{-1}$
- 3. Normalize result & check for over/underflow
  - $1.000_2 \times 2^{-4}$, with no over/underflow
- 4. Round and renormalize if necessary
  - $1.000_2 \times 2^{-4}$ (no change) = 0.0625

# FP Adder Hardware

- Much more complex than integer adder
- Doing it in one clock cycle would take too long
  - Much longer than integer operations
  - Slower clock would penalize all instructions
- FP adder usually takes several cycles
  - Can be pipelined

# FP Adder Hardware



# FP Arithmetic Hardware

- FP multiplier is of similar complexity to FP adder
  - But uses a multiplier for significands instead of an adder
- FP arithmetic hardware usually does
  - Addition, subtraction, multiplication, division, reciprocal, square-root
  - FP ↔ integer conversion
- Operations usually takes several cycles
  - Can be pipelined

# FP Instructions in MIPS

- FP hardware is coprocessor 1
  - Adjunct processor that extends the ISA
- Separate FP registers
  - 32 single-precision: $f0, $f1, … $f31
  - Paired for double-precision: $f0/$f1, $f2/$f3, …
- FP load and store instructions
  - `lwc1`, `ldc1`, `swc1`, `sdc1`
  - e.g., `ldc1 $f8, 32($sp)`
- FP instructions operate only on FP registers

# FP Instructions in MIPS

- Single-precision arithmetic
  - `add.s`, `sub.s`, `mul.s`, `div.s`
  - e.g., `add.s $f0, $f1, $f6`
- Double-precision arithmetic
  - `add.d`, `sub.d`, `mul.d`, `div.d`
  - e.g., `mul.d $f4, $f4, $f6`
- Single- and double-precision comparison
  - `c.xx.s`, `c.xx.d` (`xx=eq`, `lt`, `le`, `gt`, `ge`)
  - Sets or clears FP condition-code bit
  - e.g. `c.lt.s $f3, $f4`
- Branch on FP condition code true or false
  - `bc1t`, `bc1f`
  - e.g., `bc1t TargetLabel`